AUTOM-98 LAB: Terraform for Dynatrace

> **Series:** AUTOM — Dynatrace Automation | **Reference:** 98 — Terraform Hands-On LAB | **Created:** April 2026 | **Last Updated:** 10/29/2026

Overview

Hands-on lab for installing the Dynatrace Terraform provider, configuring authentication, creating and managing Dynatrace resources as code, importing existing configuration, and integrating into a CI/CD pipeline with GitHub Actions.

Table of Contents

1. Install Terraform
2. Configure the Dynatrace Provider
3. Create Your First Resource — a Notebook Document
4. Create Settings 2.0 Resources
5. Create IAM Resources (OAuth Required)
6. Import Existing Resources
7. Variables and Multi-Environment
8. State Management
9. GitHub Actions CI/CD Pipeline
10. Drift Detection
11. Summary and Checklist

Prerequisites

Requirement	Details
Completed	AUTOM-04: Terraform Provider (lecture notebook)
Dynatrace Environment	SaaS tenant (Gen3)
Terraform CLI	Version 1.5+ (the §6 import walkthrough uses `import` blocks); 1.11+ for native S3 state locking (§8)
Authentication	Platform Token for §3–§4; OAuth client (client ID, secret, account UUID) for §5; classic API Token only for classic resources
Permissions	Settings write, IAM admin (for Section 5)
Git	Git CLI and a GitHub account (for Section 9)

1. Install Terraform

Terraform is a single binary. Download it from terraform.io or use a package manager.

macOS (Homebrew):

```
brew tap hashicorp/tap
brew install hashicorp/tap/terraform
```

Linux (APT — Debian/Ubuntu):

```
wget -O- https://apt.releases.hashicorp.com/gpg | sudo gpg --dearmor -o
/usr/share/keyrings/hashicorp-archive-keyring.gpg
echo "deb [signed-by=/usr/share/keyrings/hashicorp-archive-keyring.gpg]
https://apt.releases.hashicorp.com $(lsb_release -cs) main" | sudo tee
/etc/apt/sources.list.d/hashicorp.list
sudo apt-get update &amp;amp;&amp;amp; sudo apt-get install terraform
```

Windows (Chocolatey):

```
choco install terraform
```

Verify the installation:

```
terraform version
# Expected output: Terraform v1.x.x
```

> **Note:** The Dynatrace Terraform provider requires Terraform 1.0 or later. This LAB uses `import` blocks (§6, Terraform 1.5+) and native S3 locking (§8, Terraform 1.11+), so install a current 1.x release.

---

2. Configure the Dynatrace Provider

Create a project directory and a `main.tf` file:

```
mkdir dynatrace-terraform &amp;amp;&amp;amp; cd dynatrace-terraform
touch main.tf
```

Add the provider block to `main.tf`:

```
terraform {
  required_providers {
    dynatrace = {
      source  = "dynatrace-oss/dynatrace"
      version = "~&gt; 1.96"
    }
  }
}
```

> **Token-type decision (read first):** The provider has a **mixed-auth model** — some resources need a Platform Token (`dt0s16`), others need a classic API Token (`dt0c01`), and IAM resources need OAuth. For the full per-resource decision matrix, see **AUTOM-04 § 3 "Service User Credentials for Terraform — Platform Token vs Classic API Token"**. The methods below cover the three credential types you will configure; the lecture explains *which one each resource needs*.

Authentication Method 1: Platform Token (default for most new resources)

Best for Settings 2.0 and most Gen3 resources (workflows, documents, segments, Davis anomaly detectors). The OpenPipeline v2 resources also accept it — the provider recommends platform credentials there because they set the settings object's owner — and Grail buckets go through the same platform client as documents. `dynatrace_platform_slo` and IAM need an OAuth client instead (Method 3). Mint in the Dynatrace UI under **Account Management > Identity & access management > Platform tokens** with the scopes the resource requires.

```
provider "dynatrace" {
  dt_env_url       = "https://&lt;env-id&gt;.apps.dynatrace.com"
  platform_token   = var.dt_platform_token
}
```

Methods 1–3 read their credentials from variables, and §5 needs the account UUID as a variable, so declare them once in `variables.tf` (§7 adds to this file):

```
variable "dt_platform_token" {
  type      = string
  sensitive = true
  default   = null
}

variable "dt_api_token" {
  type      = string
  sensitive = true
  default   = null
}

variable "dt_client_id" {
  type      = string
  sensitive = true
  default   = null
}

variable "dt_client_secret" {
  type      = string
  sensitive = true
  default   = null
}

variable "dt_account_id" {
  type        = string
  default     = null
  description = "Dynatrace account UUID — required by the IAM resources in §5"
}
```

A `null` default leaves the provider argument unset, so the provider falls back to the matching environment variable (Method 4).

Authentication Method 2: Classic API Token (legacy + a few specific resources)

Required for Synthetic monitors, the classic SLO resources (`dynatrace_slo`, `dynatrace_slo_v2`), and the `dynatrace_api_token` resource itself. Often configured **alongside** a Platform Token in the same provider block.

```
provider "dynatrace" {
  dt_env_url       = "https://&lt;env-id&gt;.live.dynatrace.com"
  dt_api_token     = var.dt_api_token
  platform_token   = var.dt_platform_token
}
```

> **Important (v1.88.0+):** The OAuth functionality was removed from ~16 provider resources in v1.88.0. Synthetic monitors, the original `dynatrace_slo` resource, and `dynatrace_api_token` no longer accept OAuth — they need a classic API Token. (The modern `dynatrace_platform_slo` is the opposite: OAuth client only.) See AUTOM-04 § 3 for the full list.

Authentication Method 3: OAuth Client Credentials (IAM, platform SLOs)

Required for `dynatrace_iam_*` resources (groups, policies, bindings) and for `dynatrace_platform_slo`. The provider docs state that *"Platform tokens can't be used for IAM (Account Management) or classic resources"*, and the SLO service is not among the services the Platform tokens (DT docs) page lists as covered. The argument names are `client_id`, `client_secret` and `account_id` — there are no `dt_`-prefixed variants (provider schema (Dynatrace GitHub)).

```
provider "dynatrace" {
  dt_env_url       = "https://&lt;env-id&gt;.apps.dynatrace.com"
  client_id        = var.dt_client_id
  client_secret    = var.dt_client_secret
  account_id       = var.dt_account_id
}
```

Authentication Method 4: Environment Variables

Keep credentials out of `.tf` files entirely:

```
export DYNATRACE_ENV_URL="https://&lt;env-id&gt;.apps.dynatrace.com"
export DT_PLATFORM_TOKEN="dt0s16.xxx..."
export DYNATRACE_API_TOKEN="dt0c01.xxx..."   # classic resources only
# OAuth client for §5 (IAM) and dynatrace_platform_slo
export DT_CLIENT_ID="dt0s02.xxx..."
export DT_CLIENT_SECRET="dt0s02.xxx.yyy..."
export DT_ACCOUNT_ID="&lt;account-uuid&gt;"
export TF_VAR_dt_account_id="$DT_ACCOUNT_ID"  # the §5 resources also take it as an argument
```

With environment variables set, the provider block needs no arguments:

```
provider "dynatrace" {}
```

> **Production callout — run Terraform as a Service User.** Outside this LAB, mint the Platform Token (and any classic API Token) on a **Service User**, not on a human's account. Three things must align: the Service User's IAM permissions, the creator's `iam:service-users:use` permission, and the token scopes at creation time. The `dynatrace_api_token` resource also stores its minted token plain-text in state — see **AUTOM-04 § 3 "Operational Safety — State File Leakage"** and the **"Bridging the Trust Boundary"** subsection for the recommended *mint-out-of-band, deliver-in-band* pattern.

Initialize the Provider

Run `terraform init` to download the provider plugin:

```
terraform init
# Initializing provider plugins...
# - Installing dynatrace-oss/dynatrace v1.105.x...   (the newest 1.x release)
# Terraform has been successfully initialized!
```

> **Provider version currency (checked 10/02/2026):** the `~> 1.96` constraint **floats** — `terraform init` will pull the newest 1.x release (v1.105.0, released 09/23/2026, at time of writing), and releases v1.97–v1.105 include stricter validation and breaking changes (`dynatrace_kubernetes_enrichment` field removal in v1.100; stricter

> OpenPipeline-v2, anomaly, and RUM validation in v1.97; legacy HTTP client and `DYNATRACE_HTTP_RESPONSE` removed in v1.101) plus new resources (`dynatrace_maintenance_windows` in v1.98 — deprecates `dynatrace_maintenance`; OpenPipeline `*_dataforwarding` in v1.99). The HCL blocks in §2–§8 were re-checked with `terraform validate` against v1.105.0 on 10/02/2026, with a resource present so that the provider blocks are checked too. If you need that validated baseline exactly, pin `version = "1.105.0"` (exact); if you float, review the provider release notes for the versions `init` selects before applying.

3. Create Your First Resource — a Notebook Document

The first resource is a Dynatrace **notebook**, managed as a `dynatrace_document`. It is a good first target: it runs on the Platform Token from Method 1 (or Method 4's `DT_PLATFORM_TOKEN`), it is private to you, it changes nothing about how your tenant monitors or alerts, and you can open it in the Notebooks app a few seconds after `apply`.

Add the resource to `main.tf`:

```
resource "dynatrace_document" "first_notebook" {
  type    = "notebook"
  name    = "Terraform LAB - first notebook"
  private = true # visible only to the token's owner
  content = jsonencode({
    version = "7"
    sections = [
      {
        id       = "intro"
        type     = "markdown"
        markdown = "## Managed by Terraform\nEdits made in the app will show
up as drift on the next `terraform plan`."
      },
      {
        id    = "errors"
        type  = "dql"
        title = "Error logs, last hour"
        state = {
          input = {
            value = "fetch logs, from:-1h\n| filter status == \"ERROR\"\n|
summarize errors = count()"
          }
        }
      }
    ]
  })
}
```

Two schema details that trip people up: `version` is the **string** `"7"`, and a section has **no `content` key** — a `markdown` section carries `markdown`, a `dql` section carries its query under `state.input.value` (AUTOM-04 §4 *Grail Notebook*).

Preview the Change

```
terraform plan
```

Expected output (abbreviated — the `content` attribute prints as the full `jsonencode(...)` body):

```
Terraform will perform the following actions:

  # dynatrace_document.first_notebook will be created
  + resource "dynatrace_document" "first_notebook" {
      + content = jsonencode(
            {
              + sections = [ ... ]
              + version  = "7"
            }
        )
      + id      = (known after apply)
      + name    = "Terraform LAB - first notebook"
      + private = true
      + type    = "notebook"
      ...
    }

Plan: 1 to add, 0 to change, 0 to destroy.
```

Apply the Change

```
terraform apply
# Type "yes" when prompted to confirm
```

Verify the Resource

```
terraform show
```

Open the **Notebooks** app and look for *Terraform LAB - first notebook*. Now edit its markdown section in the app, run `terraform plan` again, and watch Terraform report the edit as a change it wants to revert — that is drift detection (§10) in miniature.

> **Classic equivalent — earlier versions of this LAB.** This step used to create an alerting profile (`dynatrace_alerting`). That resource still works on a tenant that has not been upgraded, but it is Dynatrace Classic: `builtin:alerting.profile` is on the removed-schemas list (DT docs), and its Gen3 successor is a problem-triggered `dynatrace_automation_workflow` (AUTOM-04 §4, §6). If your tenant still runs alerting profiles and you need to manage them until the upgrade, this is the shape (the same plan → apply → show loop applies):

```
hcl &gt; # Dynatrace Classic — unupgraded tenants only; classic API token (settings.read / settings.write) &gt; resource "dynatrace_alerting" "production" { &gt; name = "Production Alerts" &gt; rules { &gt; rule { &gt; include_mode = "NONE" # NONE | INCLUDE_ALL | INCLUDE_ANY &gt; severity_level = "AVAILABILITY" &gt; delay_in_minutes = 0 &gt; } &gt; rule { &gt; include_mode = "NONE" &gt; severity_level = "ERRORS" &gt; delay_in_minutes = 5 # successor: problem_open_duration on the workflow trigger &gt; } &gt; } &gt; } &gt;
```

---

4. Create Settings 2.0 Resources

The `dynatrace_generic_setting` resource can manage any Settings 2.0 schema. This is useful when the provider does not have a dedicated resource type for a particular setting.

Example: Ownership Team

`builtin:ownership.teams` is a multi-object schema (so creating one never overwrites an existing object) and survives the upgrade to Latest Dynatrace. The value below passes the Settings API's validate-only check (09/18/2026).

```
resource "dynatrace_generic_setting" "platform_team" {
  schema = "builtin:ownership.teams"
  scope  = "environment"
  value = jsonencode({
    name        = "Platform Team"
    identifier  = "platform-team"
    description = "Owned by the Terraform LAB"
    responsibilities = {
      development    = false
      infrastructure = true
      lineOfBusiness = false
      operations     = true
      security       = false
    }
    supplementaryIdentifiers = []
    contactDetails           = []
    links                    = []
    additionalInformation    = []
  })
}
```

Key Attributes

Attribute	Description
`schema`	The Settings 2.0 schema identifier (e.g., `builtin:ownership.teams`). The argument is `schema` — `schema_id` fails `terraform validate`
`scope`	Where the setting applies: `environment`, `HOST-xxx`, `KUBERNETES_CLUSTER-xxx`, etc.
`value`	JSON-encoded configuration object matching the schema definition

Finding Schema IDs

Navigate to **Settings** in the Dynatrace UI. The schema ID appears in the URL when you open any setting:

```
https://&lt;env-id&gt;.apps.dynatrace.com/ui/settings/builtin:ownership.teams
                                            ^^^^^^^^^^^^^^^^^^^^^^^
                                            This is the schema
```

Before choosing a schema for new automation, check it is not on the removed-schemas list (DT docs); `builtin:kubernetes.generic.metadata.enrichment`, used in earlier versions of this LAB, is a deprecated singleton whose rule fields differ from what was shown here.

Alternatively, use the Settings API:

```
curl -H "Authorization: Api-Token $DT_API_TOKEN" \
  "https://&lt;env-id&gt;.live.dynatrace.com/api/v2/settings/schemas" | jq
'.items[].schemaId'
```

---

5. Create IAM Resources (OAuth Required)

IAM resources (groups, policies, bindings) require OAuth client credentials (Method 3, or the `DT_CLIENT_ID` / `DT_CLIENT_SECRET` / `DT_ACCOUNT_ID` environment variables of Method 4). Neither API tokens nor Platform Tokens can manage IAM.

> **Deeper IAM hands-on:** This section gives the minimum viable IAM pattern alongside the main Terraform LAB. For the full IAM lifecycle — OAuth client setup with four minimal scopes, permission-statement lookup (the IAM policy reference) and account discovery, the four IAM resource types, boundaries, the `bindings_v2` re-assigns-all caveat, deprecated arguments to avoid, bulk export of an existing account, and HTTP 400 troubleshooting — see **AUTOM-95 LAB: Terraform IAM Management**.

Create an IAM Group

```
resource "dynatrace_iam_group" "sre_team" {
  name = "SRE Team"
}
```

Create an IAM Policy

```
resource "dynatrace_iam_policy" "sre_policy" {
  name            = "SRE Environment Access"
  account         = var.dt_account_id
  statement_query = "ALLOW environment:roles:viewer;"
  tags            = ["sre"]
}
```

Bind the Policy to the Group

```
resource "dynatrace_iam_policy_bindings_v2" "sre_binding" {
  group   = dynatrace_iam_group.sre_team.id
  account = var.dt_account_id

  policy {
    id = dynatrace_iam_policy.sre_policy.id
  }
}
```

Apply

```
terraform plan
terraform apply
```

> **Note:** Both the policy and the binding take `account` — your Dynatrace account UUID, found under **Account Management > Account settings** (passed here as `var.dt_account_id`, declared in §2). `environment:roles:viewer` *"Grants user the Access environment permission"* (IAM policy statements reference (DT docs)) — the policy name says what it grants. A policy needs exactly one of `account` / `environment` (environment-level policies are deprecated). The v2 binding takes one `group` and one `policy { id = … }` block per bound policy, and it **re-assigns every policy on that group** — list all of them.

---

6. Import Existing Resources

If you already have Dynatrace resources configured manually, you can bring them under Terraform management without recreating them.

> **Bootstrapping from an existing tenant in bulk?** Use the provider's built-in `-export` **utility** rather than running `terraform import` per-resource. From a downloaded provider binary: `./terraform-provider-dynatrace -export -ref -id` (canonical form per Terraform CLI commands (DT docs) — `-ref` emits inter-resource references rather than hardcoded IDs, `-id` adds commented IDs for traceability; supply credentials via env vars per AUTOM-04 §3). Generates `.tf` files for the entire tenant. The per-resource flow below is right when you only need to onboard a few specific resources.

The walk-through below imports an existing **workflow**; the same steps apply to any resource type.

Step 1: Find the Resource ID

Locate the resource ID from the Dynatrace UI or API. For a workflow, open it in the Workflows app — the ID is the last segment of the URL:

```
https://&lt;env-
id&gt;.apps.dynatrace.com/ui/apps/dynatrace.automations/workflows/abc12345-
def6-7890-abcd-ef1234567890

^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^

This is the resource ID
```

Step 2: Add an `import` Block

Add an `import` block — not a resource block — to a `.tf` file:

```
import {
  to = dynatrace_automation_workflow.existing_workflow
  id = "abc12345-def6-7890-abcd-ef1234567890"
}
```

HashiCorp documents this as the alternative to the CLI command: *"Instead of manually importing resources, you can add the import block to your Terraform configurations so that Terraform imports resources when you run the terraform apply command."* (terraform import (HashiCorp))

Step 3: Generate the HCL

```
terraform plan -generate-config-out=generated.tf
```

Terraform writes a `resource "dynatrace_automation_workflow" "existing_workflow"` block to `generated.tf` (a file that must not already exist). HashiCorp marks this as experimental: *"Configuration generation is available in Terraform v1.5 as an experimental feature."* (Generating configuration (HashiCorp)) Treat the output as a draft — remove attributes you do not want to own, and replace hard-coded IDs with references.

Step 4: Apply the Import

```
terraform apply
```

The apply records the workflow in state without changing it. Afterwards you can delete the `import` block.

> **CLI alternative.** `terraform import dynatrace_automation_workflow.existing_workflow <id>` also works, but it needs a resource block with that address to exist first, and it writes no configuration. Do not save `terraform show` output next to that block as `imported.tf` : it declares the same address a second time, so the next plan fails, and it is a rendering of state rather than HCL you can apply. Fill in the existing block by hand instead.

Common Import Targets

Resource Type	ID Source
`dynatrace_automation_workflow`	Workflows app URL
`dynatrace_document` (dashboards, notebooks)	Dashboards / Notebooks app URL, or the Documents API

Resource Type	ID Source
`dynatrace_platform_slo`	SLO app or the SLO Service Public API — and name it explicitly when exporting, it is excluded by default
`dynatrace_segment`	Segments UI or `-export dynatrace_segment`
`dynatrace_generic_setting`	Settings API (`objectId` field)
Classic: `dynatrace_alerting` , `dynatrace_json_dashboard` , `dynatrace_slo_v2`	Settings UI URL / classic dashboard URL / SLO API — only while the tenant still runs these (AUTOM-04 §4 *Classic Resources*)

> **Tip:** After import, run `terraform plan` to verify the imported state matches the live configuration. Any differences indicate drift that you should reconcile in the `.tf` file.

---

7. Variables and Multi-Environment

Define Variables

Add to the `variables.tf` you started in §2:

```
variable "dt_env_url" {
  type        = string
  description = "Dynatrace environment URL"
}

variable "environment" {
  type        = string
  default     = "dev"
  description = "Target environment name (dev, staging, production)"
}
```

Supply Values with tfvars

Create `terraform.tfvars` (add to `.gitignore` — never commit credentials):

```
dt_env_url        = "https://&lt;env-id&gt;.apps.dynatrace.com"
dt_platform_token = "dt0s16.xxx..."
dt_account_id     = "&lt;account-uuid&gt;"
environment       = "production"
```

Approach 1: Directory-Based Multi-Environment (recommended)

Each Dynatrace environment is a separate tenant with its own credentials, so give each one its own root directory, with shared modules:

```
dynatrace-terraform/
  modules/
    problem-routing/
      main.tf
      variables.tf
  environments/
    dev/
      main.tf         # calls module with dev values
      terraform.tfvars
    staging/
      main.tf
      terraform.tfvars
    production/
      main.tf
      terraform.tfvars
```

Each environment directory runs independently with its own state, credentials and variables. AUTOM-09 §2 and §6 build this layout out.

Approach 2: Workspaces — same tenant only

Terraform workspaces keep separate state files for one configuration and one set of credentials. HashiCorp is explicit that they do not fit separate tenants: *"Workspaces are not appropriate for system decomposition or deployments requiring separate credentials and access controls."* (Workspaces (HashiCorp)) Use them only for disposable copies inside **one** tenant — for example, a per-feature test notebook:

```
terraform workspace new feature-x
terraform workspace select feature-x
# terraform.workspace returns the current workspace name
```

```
resource "dynatrace_document" "env_notebook" {
  type    = "notebook"
  name    = "${terraform.workspace} - runbook"
  content = file("${path.module}/notebooks/runbook.json")
}
```

---

8. State Management

Terraform state tracks the mapping between your `.tf` files and real Dynatrace resources. By default, state is stored locally in `terraform.tfstate`.

> **State-file leakage warning.** If your Terraform manages the `dynatrace_api_token` resource, the minted token value is written **plain-text** to state regardless of `sensitive = true`. The same risk applies to any provider that produces a credential at apply time. Mitigate with backend encryption + restricted access — see **AUTOM-04 § 3 "Operational Safety — State File Leakage When Minting Tokens"** for the full pattern, and **AUTOM-09 § 3** (state backends) and **§ 8** (secrets handling) for the bootstrap recipe.

Local State (Default)

Works for single-operator setups. The state file is created automatically after `terraform apply`.

> **Warning:** Never commit `terraform.tfstate` to version control. It may contain sensitive values. Add it to `.gitignore`.

Remote State — S3 Backend

For team use, store state in a shared remote backend. The S3 backend supports **native locking with a lockfile** (`use_lockfile`) — generally available since Terraform 1.11 (experimental in 1.10); the Terraform 1.11 changelog (HashiCorp GitHub) states *"S3 native state locking is now generally available."* No DynamoDB table is required —

HashiCorp's S3 backend (HashiCorp) page says *"DynamoDB-based locking is deprecated and will be removed in a future minor version."*

```
terraform {
  backend "s3" {
    bucket       = "my-terraform-state"
    key          = "dynatrace/terraform.tfstate"
    region       = "us-east-1"
    encrypt      = true
    use_lockfile = true   # native S3 locking (GA in Terraform 1.11); DynamoDB
locking is deprecated
  }
}
```

Attribute	Purpose
`bucket`	S3 bucket for state storage
`key`	Path within the bucket
`use_lockfile`	Native S3 lockfile-based concurrency control (GA in Terraform 1.11); replaces the deprecated DynamoDB-based locking
`encrypt`	Encrypt state at rest (use SSE-KMS with a customer-managed key for production)

> **Migrating from DynamoDB?** Set `use_lockfile = true` alongside the existing `dynamodb_table` attribute, run `terraform init -reconfigure`, verify lock acquisition succeeds, then remove `dynamodb_table` on a follow-up apply. See AUTOM-09 § 3 for the full migration recipe.

Remote State — Azure Blob

```
terraform {
  backend "azurerm" {
    resource_group_name  = "rg-terraform"
    storage_account_name = "tfstateaccount"
    container_name       = "tfstate"
    key                  = "dynatrace.terraform.tfstate"
  }
}
```

Remote State — Terraform Cloud / HCP Terraform

```
terraform {
  cloud {
    organization = "my-org"
    workspaces {
      name = "dynatrace-production"
    }
  }
}
```

Inspecting State

```
# List all resources in state
terraform state list

# Show details of a specific resource
terraform state show dynatrace_document.first_notebook

# Remove a resource from state (without destroying it)
terraform state rm dynatrace_document.first_notebook
```

---

9. GitHub Actions CI/CD Pipeline

This section is a **minimal starter** to confirm the basics work end-to-end with static GitHub Secrets. For production CI/CD — OIDC-federated secret retrieval, multi-environment promotion, plan-comments on PRs, manual-approval gates, and patterns for GitLab / Bitbucket / Bamboo / Jenkins / Azure DevOps — go to:

Where	What it covers
AUTOM-96 LAB: GitHub Actions CI/CD for Dynatrace Terraform	Full hands-on: OIDC → Vault → Platform Token, plan-comment via `actions/github-script`, environment-gated apply, validation walkthrough. The recommended-default end state.
AUTOM-07: CI/CD Integration	Conceptual coverage of GitHub Actions, GitLab CI, Bitbucket Pipelines, Atlassian Bamboo, Jenkins, Azure DevOps.

Where	What it covers
AUTOM-04 § 3 "Bridging the Trust Boundary"	Why static GitHub Secrets is no longer the recommended default, and the five mechanisms for getting tokens to the runner.

The starter below uses static secrets and is intentionally kept simple — do **not** ship this pattern to production. It passes every credential this LAB's resources use: the Platform Token for the §3 notebook and §4 setting, and the OAuth client for the §5 IAM resources. Drop the ones your configuration does not need.

Starter Workflow (static secrets — not for production)

Create `.github/workflows/terraform-deploy.yml`:

```yaml
name: Deploy Dynatrace Terraform (starter)

on:
  pull_request:
    paths: ["terraform/**"]
  push:
    branches: [main]
    paths: ["terraform/**"]

env:
  DYNATRACE_ENV_URL: ${{ secrets.DT_ENV_URL }}
  DYNATRACE_PLATFORM_TOKEN: ${{ secrets.DT_PLATFORM_TOKEN }}
  # OAuth client: IAM resources (§5) and dynatrace_platform_slo
  DT_CLIENT_ID: ${{ secrets.DT_CLIENT_ID }}
  DT_CLIENT_SECRET: ${{ secrets.DT_CLIENT_SECRET }}
  DT_ACCOUNT_ID: ${{ secrets.DT_ACCOUNT_ID }}
  TF_VAR_dt_account_id: ${{ secrets.DT_ACCOUNT_ID }}
  # Classic API token — only if you manage classic resources
  # DYNATRACE_API_TOKEN: ${{ secrets.DT_API_TOKEN }}

jobs:
  plan:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v7
      - uses: hashicorp/setup-terraform@v4

      - name: Terraform Init
        run: terraform init
        working-directory: terraform/

      - name: Terraform Plan
        run: terraform plan -no-color
        working-directory: terraform/

  apply:
    if: github.ref == 'refs/heads/main' &amp;&amp; github.event_name == 'push'
    needs: plan
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v7
      - uses: hashicorp/setup-terraform@v4

      - name: Terraform Init
        run: terraform init
        working-directory: terraform/
```

```
      - name: Terraform Apply
        run: terraform apply -auto-approve
        working-directory: terraform/
```

Action versions are the latest majors as of 10/2026 (`actions/checkout` v7, `hashicorp/setup-terraform` v4 — v4 *"requires Node.js 24"*, per its v4.0.0 release notes (HashiCorp GitHub)).

GitHub Secrets (starter only)

Store credentials in GitHub repository secrets (**Settings > Secrets and variables > Actions**):

Secret Name	Value
`DT_ENV_URL`	`https://<env-id>.apps.dynatrace.com`
`DT_PLATFORM_TOKEN`	Platform Token with the scopes your resources need (§2, Method 1)
`DT_CLIENT_ID` / `DT_CLIENT_SECRET`	OAuth client for IAM and platform SLOs (§2, Method 3)
`DT_ACCOUNT_ID`	Your account UUID
`DT_API_TOKEN`	Classic API token — only if you manage classic resources

> **Production migration path:** Once the starter runs cleanly, walk through **AUTOM-96 LAB** to replace these static secrets with OIDC-federated runtime credential fetch from Vault (or AWS Secrets Manager / Azure Key Vault / GCP Secret Manager). The end state has zero long-lived Dynatrace credentials in GitHub Secrets.

---

10. Drift Detection

Drift occurs when someone changes a Dynatrace resource manually (through the UI or API) after it was deployed via Terraform.

Detect Drift

```
# Run plan with detailed exit code
terraform plan -detailed-exitcode

# Exit codes:
#   0 = No changes (no drift)
#   1 = Error
#   2 = Changes detected (drift found)
```

Scheduled Drift Detection

Add a scheduled GitHub Actions workflow to check for drift daily. It branches on the exit code, so that an error (exit 1 — an expired token, for example) fails the job instead of being reported as drift:

```yaml
name: Drift Detection

on:
  schedule:
    - cron: "0 8 * * 1-5"   # Weekdays at 8 AM UTC

jobs:
  detect-drift:
    runs-on: ubuntu-latest
    env:
      DYNATRACE_ENV_URL: ${{ secrets.DT_ENV_URL }}
      DYNATRACE_PLATFORM_TOKEN: ${{ secrets.DT_PLATFORM_TOKEN }}
      DT_CLIENT_ID: ${{ secrets.DT_CLIENT_ID }}
      DT_CLIENT_SECRET: ${{ secrets.DT_CLIENT_SECRET }}
      DT_ACCOUNT_ID: ${{ secrets.DT_ACCOUNT_ID }}
      TF_VAR_dt_account_id: ${{ secrets.DT_ACCOUNT_ID }}
    steps:
      - uses: actions/checkout@v7
      - uses: hashicorp/setup-terraform@v4
        with:
          terraform_wrapper: false   # plain exit codes
      - run: terraform init
        working-directory: terraform/
      - name: Check for drift
        id: drift
        working-directory: terraform/
        run: |
          set +e
          terraform plan -detailed-exitcode -no-color
          echo "rc=$?" &gt;&gt; "$GITHUB_OUTPUT"
      - name: Fail on plan error
        if: steps.drift.outputs.rc == '1'
        run: |
          echo "terraform plan failed - this is an error, not drift."
          exit 1
      - name: Alert on drift
        if: steps.drift.outputs.rc == '2'
        run: echo "Drift detected! Review terraform plan output."
```

Handling Drift

Scenario	Action
Manual change was intentional	Update `.tf` to match, or import the new resource (§6)

Scenario	Action
Manual change was accidental	Run `terraform apply` to restore the desired state
State is stale	Run `terraform apply -refresh-only`, review the proposed state changes, then confirm. `terraform refresh` is deprecated: *"Instead, add the -refresh-only flag to terraform apply and terraform plan commands."* (terraform refresh (HashiCorp))

---

11. Summary and Checklist

Deployment Checklist

- [ ] Terraform 1.5+ installed and verified (1.11+ for native S3 backend locking)
- [ ] Provider configured with the **right token type per resource** (Platform / classic API / OAuth — see AUTOM-04 § 3)
- [ ] `terraform init` downloads the Dynatrace provider (v1.105.x at time of writing)
- [ ] Resources defined in `.tf` files, not configured manually
- [ ] `terraform.tfvars` and `terraform.tfstate` in `.gitignore`
- [ ] Remote state backend configured for team use, with encryption at rest
- [ ] **Production:** token holder is a Service User, not a human account (see AUTOM-04 § 3)
- [ ] **Production:** no `dynatrace_api_token` minted in this state file (or backend access is locked down — state stores the token plain-text)
- [ ] CI/CD pipeline runs `plan` on PRs, `apply` on merge — production uses OIDC-federated secret fetch (see AUTOM-96)
- [ ] Drift detection scheduled

Key Takeaways

Concept	Key Point
Provider Setup	Pin version with `~> 1.96` (or `~> 1.105` for current resources); v1.88.0+ uses a mixed-auth model

Concept	Key Point
Authentication	Platform Token (default, incl. OpenPipeline v2 and buckets) + classic API Token (Synthetics, classic SLOs, `dynatrace_api_token`) + OAuth (IAM, `dynatrace_platform_slo`) — see AUTOM-04 § 3
Service User	Production token holder is a Service User; three things must align (perms, creator scope, token scope)
Plan Before Apply	Always review `terraform plan` output before applying
State Management	Use remote backends with encryption; `dynatrace_api_token` stores its value plain-text in state regardless of `sensitive`
Import	Use `-export` utility for bulk bootstrap; `import` blocks for individual resources
CI/CD	Static GitHub Secrets is a starter; production uses OIDC → Vault (or cloud-native secret manager) — see AUTOM-96
Drift Detection	Schedule regular checks with `plan -detailed-exitcode`

Monaco vs Terraform — When to Use Which

Scenario	Recommended Tool
Quick config export/import	Either (Monaco UI flow, or Terraform `-export` utility)
Multi-environment promotion	Either
IAM policy management	Terraform (OAuth required)
Multi-cloud infrastructure + Dynatrace	Terraform
Drift detection and state tracking	Terraform
Team with existing Terraform expertise	Terraform
Team new to IaC	Monaco (lower learning curve)

References

Resource	URL
Terraform Registry	registry.terraform.io/providers/dynatrace-oss/dynatrace
Provider Documentation	registry.terraform.io/providers/dynatrace-oss/dynatrace/latest/docs
Provider GitHub	github.com/dynatrace-oss/terraform-provider-dynatrace
Terraform Install	developer.hashicorp.com/terraform/install
Dynatrace IAM Docs	docs.dynatrace.com/docs/manage/identity-access-management

---

*Continue to **AUTOM-09: Terraform GitOps Setup Recipe** for state backends, lifecycle protections, and multi-environment promotion; then **AUTOM-07: CI/CD Integration** for production pipeline patterns (and the **AUTOM-96 LAB** for the GitHub Actions hands-on); or jump to **AUTOM-05: Dynatrace Workflows** for event-driven automation.*

Community Resources

Repository	Description
terraform-provider-dynatrace	Official provider (v1.105.x at time of writing) with built-in `-export` utility
community-examples/configuration-as-code	Starter templates in `basic-templates-terraform` (moved from the archived `dynatrace-configuration-as-code-samples` repo), plus modules, IAM, and DQL examples
terraform_modules	Reusable module pattern for synthetic monitors
terraform_dql_example	DQL as a Terraform data source for dynamic config generation
terraform_team_onboarding	IAM policies, groups, and Azure Entra ID for team provisioning
iam_tf_sample	IAM policies, Grail buckets, OpenPipelines, and segments

> **Tip:** Use the provider export feature (`./terraform-provider-dynatrace -export -ref -id` — canonical Dynatrace-recommended form; supply credentials via env vars per AUTOM-04 §3) to generate `.tf` files from an existing tenant — the fastest path to Terraform-managed configuration. See AUTOM-04 §8 for the full flag reference.

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.